

# **CSS Fullscreen Pseudo-class**

Complete Guide: From Basics to Advanced

CSS Guide

May 10, 2026

# Contents

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                 | <b>2</b>  |
| 1.1      | What is the Fullscreen Pseudo-class? . . . . .      | 2         |
| 1.2      | Fullscreen vs. Regular Display . . . . .            | 2         |
| 1.2.1    | Key Differences . . . . .                           | 2         |
| 1.3      | Why Use :fullscreen? . . . . .                      | 2         |
| <b>2</b> | <b>Core Concepts</b>                                | <b>4</b>  |
| 2.1      | Understanding the Fullscreen Pseudo-class . . . . . | 4         |
| 2.1.1    | Basic Definition . . . . .                          | 4         |
| 2.1.2    | Browser Compatibility . . . . .                     | 4         |
| 2.2      | Related Pseudo-classes . . . . .                    | 4         |
| 2.2.1    | :fullscreen vs. :fullscreen-backdrop . . . . .      | 4         |
| <b>3</b> | <b>Syntax &amp; Usage</b>                           | <b>6</b>  |
| 3.1      | Basic Syntax . . . . .                              | 6         |
| 3.1.1    | Simple Pseudo-class Usage . . . . .                 | 6         |
| 3.1.2    | Vendor Prefixes . . . . .                           | 6         |
| 3.1.3    | Complete Cross-Browser Implementation . . . . .     | 7         |
| <b>4</b> | <b>The Fullscreen API</b>                           | <b>8</b>  |
| 4.1      | Requesting Fullscreen . . . . .                     | 8         |
| 4.1.1    | Basic Fullscreen Request . . . . .                  | 8         |
| 4.1.2    | Complete Video Player Example . . . . .             | 8         |
| <b>5</b> | <b>Practical Examples</b>                           | <b>10</b> |
| 5.1      | Example 1: Video Player Fullscreen . . . . .        | 10        |
| 5.1.1    | HTML Structure . . . . .                            | 10        |
| 5.1.2    | CSS Styling . . . . .                               | 10        |
| 5.2      | Example 2: Image Viewer Fullscreen . . . . .        | 11        |
| 5.2.1    | Complete Implementation . . . . .                   | 11        |
| 5.3      | Example 3: Game Canvas Fullscreen . . . . .         | 13        |
| 5.3.1    | Game Container Styling . . . . .                    | 13        |
| 5.4      | Example 4: Presentation Mode . . . . .              | 14        |
| 5.4.1    | Fullscreen Presentation Slides . . . . .            | 14        |

|           |                                                     |           |
|-----------|-----------------------------------------------------|-----------|
| <b>6</b>  | <b>Advanced Techniques</b>                          | <b>17</b> |
| 6.1       | Animations in Fullscreen . . . . .                  | 17        |
| 6.1.1     | Fullscreen Transition Animation . . . . .           | 17        |
| 6.2       | Responsive Fullscreen . . . . .                     | 18        |
| 6.2.1     | Fullscreen with Media Queries . . . . .             | 18        |
| 6.3       | Combining with Other Pseudo-classes . . . . .       | 18        |
| 6.3.1     | Interactive Fullscreen Styles . . . . .             | 18        |
| 6.4       | Keyboard Shortcuts in Fullscreen . . . . .          | 19        |
| 6.4.1     | JavaScript Implementation . . . . .                 | 19        |
| <b>7</b>  | <b>Browser Support</b>                              | <b>20</b> |
| 7.1       | Current Support Status . . . . .                    | 20        |
| 7.2       | Fallback Strategies . . . . .                       | 20        |
| 7.2.1     | Progressive Enhancement . . . . .                   | 20        |
| <b>8</b>  | <b>Best Practices</b>                               | <b>22</b> |
| 8.1       | Key Recommendations . . . . .                       | 22        |
| 8.1.1     | 1. Always Provide Exit Instructions . . . . .       | 22        |
| 8.1.2     | 2. Optimize Typography for Fullscreen . . . . .     | 22        |
| 8.1.3     | 3. Use Safe Area Insets . . . . .                   | 23        |
| 8.1.4     | 4. Provide Feedback on Fullscreen State . . . . .   | 23        |
| 8.1.5     | 5. Test on Multiple Devices . . . . .               | 23        |
| <b>9</b>  | <b>Common Issues &amp; Solutions</b>                | <b>24</b> |
| 9.1       | Issue 1: Fullscreen Request Denied . . . . .        | 24        |
| 9.2       | Issue 2: Styling Not Applied . . . . .              | 24        |
| 9.3       | Issue 3: Controls Disappear in Fullscreen . . . . . | 25        |
| 9.4       | Issue 4: Mobile Fullscreen Problems . . . . .       | 25        |
| <b>10</b> | <b>Advanced Use Cases</b>                           | <b>27</b> |
| 10.1      | Multi-Element Fullscreen . . . . .                  | 27        |
| 10.1.1    | Managing Multiple Fullscreen Elements . . . . .     | 27        |
| 10.1.2    | JavaScript Coordination . . . . .                   | 27        |
| 10.2      | Nested Fullscreen Elements . . . . .                | 28        |
| <b>11</b> | <b>Summary &amp; Quick Reference</b>                | <b>29</b> |
| 11.1      | Key Takeaways . . . . .                             | 29        |
| 11.1.1    | When to Use :fullscreen . . . . .                   | 29        |
| 11.1.2    | When NOT to Use . . . . .                           | 29        |
| 11.2      | Pseudo-class Reference . . . . .                    | 29        |
| 11.3      | Common Properties in Fullscreen . . . . .           | 30        |
| 11.4      | Complete CSS Template . . . . .                     | 30        |

# Chapter 1

## Introduction

### 1.1 What is the Fullscreen Pseudo-class?

The `:fullscreen` pseudo-class in CSS represents an element that is currently displayed in fullscreen mode. This powerful CSS feature allows developers to style elements differently when they're shown in fullscreen, creating optimized experiences for immersive applications like games, video players, and presentations.

```
/* Basic syntax */
element:fullscreen {
    /* Styles applied when element is in fullscreen */
}
```

When a user requests fullscreen mode (typically via the Fullscreen API), the browser switches to fullscreen presentation mode, and any CSS rules using the `:fullscreen` pseudo-class are applied.

### 1.2 Fullscreen vs. Regular Display

#### 1.2.1 Key Differences

| Aspect     | Regular Display      | Fullscreen                      |
|------------|----------------------|---------------------------------|
| Browser UI | Address bar visible  | No browser UI                   |
| Screen Use | Partial screen       | Entire screen                   |
| Dimensions | Viewport constrained | 100vw × 100vh                   |
| Styling    | Default styles       | <code>:fullscreen</code> styles |
| User Exit  | Refresh/navigate     | ESC key                         |
| Use Cases  | General browsing     | Games, videos, presentations    |

### 1.3 Why Use `:fullscreen`?

The `:fullscreen` pseudo-class is essential for:

- **Video Players:** Optimize video controls and layout for fullscreen
- **Gaming:** Maximize screen space for immersive gameplay
- **Presentations:** Hide UI elements for distraction-free presentations

- **Immersive Experiences:** AR/VR applications requiring full viewport
- **Kiosk Applications:** Display full-screen content without distraction
- **Data Visualization:** Show dashboards and charts in immersive mode

# Chapter 2

## Core Concepts

### 2.1 Understanding the Fullscreen Pseudo-class

#### 2.1.1 Basic Definition

The `:fullscreen` pseudo-class matches an element when:

1. The element has requested fullscreen mode via the Fullscreen API
2. The browser has granted the fullscreen request
3. The element is currently displayed in fullscreen mode

#### 2.1.2 Browser Compatibility

The `:fullscreen` pseudo-class requires:

- **Modern Browser:** All modern browsers support this feature
- **HTTPS:** The Fullscreen API requires a secure context (HTTPS)
- **User Gesture:** Fullscreen requests must be triggered by user interaction
- **Permissions:** Some browsers may ask for permission

### 2.2 Related Pseudo-classes

#### 2.2.1 `:fullscreen` vs. `:fullscreen-backdrop`

- `:fullscreen` - Styles the fullscreen element itself
- `::backdrop` - Styles the backdrop behind fullscreen element
- `:fullscreen-backdrop` - Legacy name for backdrop styling

```
/* Style the fullscreen element */
element:fullscreen {
  background: black;
  width: 100vw;
```

```
    height: 100vh;
}

/* Style the backdrop (the dark background) */
element::backdrop {
    background-color: rgba(0, 0, 0, 0.8);
}
```

## Chapter 3

# Syntax & Usage

### 3.1 Basic Syntax

#### 3.1.1 Simple Pseudo-class Usage

```
/* Select element in fullscreen mode */
.video-player:fullscreen {
    width: 100vw;
    height: 100vh;
    background: black;
}

/* Can combine with other selectors */
div:fullscreen {
    margin: 0;
    padding: 0;
}

/* Using with element selectors */
video:fullscreen {
    width: 100%;
    height: 100%;
}
```

#### 3.1.2 Vendor Prefixes

For broader compatibility, include vendor-prefixed versions:

```
/* Standard (modern browsers) */
element:fullscreen { }

/* Firefox */
element:-moz-full-screen { }

/* Webkit browsers (Chrome, Safari, Edge) */
element:-webkit-full-screen { }

/* IE/Edge legacy */
element:-ms-fullscreen { }
```



### 3.1.3 Complete Cross-Browser Implementation

```
video:fullscreen,  
video:-moz-full-screen,  
video:-webkit-full-screen,  
video:-ms-fullscreen {  
    width: 100vw;  
    height: 100vh;  
    background: black;  
    margin: 0;  
    padding: 0;  
}
```

## Chapter 4

# The Fullscreen API

### 4.1 Requesting Fullscreen

Before CSS `:fullscreen` styles apply, you need to request fullscreen using JavaScript.

#### 4.1.1 Basic Fullscreen Request

```
/* CSS side: styles when fullscreen is active */
.video-container:fullscreen {
  width: 100vw;
  height: 100vh;
  background: #000;
}
```

```
/* JavaScript side: request fullscreen */
const element = document.querySelector('.video-container');

// Request fullscreen
element.requestFullscreen().catch(err => {
  console.error(`Error attempting to enable fullscreen:
    ${err.message}`);
});

// Handle fullscreen change
document.addEventListener('fullscreenchange', () => {
  if (document.fullscreenElement) {
    console.log('Element is in fullscreen');
  } else {
    console.log('Exited fullscreen');
  }
});

// Exit fullscreen
document.exitFullscreen();
```

#### 4.1.2 Complete Video Player Example

```
class VideoPlayer {
  constructor(selector) {
    this.container = document.querySelector(selector);
    this.button = this.container.querySelector('.fullscreen-btn');
    this.button.addEventListener('click',
      () => this.toggleFullscreen());
  }

  toggleFullscreen() {
    if (!document.fullscreenElement) {
      this.container.requestFullscreen();
    } else {
      document.exitFullscreen();
    }
  }
}

// Usage
const player = new VideoPlayer('.video-player');
```

## Chapter 5

# Practical Examples

### 5.1 Example 1: Video Player Fullscreen

#### 5.1.1 HTML Structure

```
<div class="video-player">
  <video src="movie.mp4"></video>
  <div class="controls">
    <button class="play-btn">Play</button>
    <button class="fullscreen-btn">Fullscreen</button>
  </div>
</div>
```

#### 5.1.2 CSS Styling

```
.video-player {
  position: relative;
  width: 100%;
  max-width: 800px;
  background: black;
  margin: 20px auto;
}

video {
  width: 100%;
  height: auto;
  display: block;
}

.controls {
  position: absolute;
  bottom: 0;
  left: 0;
  right: 0;
  background: linear-gradient(transparent, rgba(0,0,0,0.7));
  padding: 20px;
  display: flex;
  gap: 10px;
```

```
}

button {
  padding: 10px 20px;
  background: rgba(255,255,255,0.2);
  color: white;
  border: 1px solid white;
  border-radius: 4px;
  cursor: pointer;
  font-size: 14px;
}

button:hover {
  background: rgba(255,255,255,0.3);
}

/* Fullscreen styles */
.video-player:fullscreen {
  width: 100vw;
  height: 100vh;
  max-width: none;
  margin: 0;
  border-radius: 0;
}

.video-player:fullscreen video {
  width: 100%;
  height: 100%;
}

.video-player:fullscreen .controls {
  font-size: 18px;
}

.video-player:fullscreen button {
  padding: 15px 25px;
  font-size: 16px;
}

/* Backdrop styling */
.video-player::backdrop {
  background-color: rgba(0, 0, 0, 0.95);
}
```

## 5.2 Example 2: Image Viewer Fullscreen

### 5.2.1 Complete Implementation

```
/* Default image gallery view */
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
}
```

```
    gap: 20px;
    padding: 20px;
    background: #f5f5f5;
}

.gallery-item {
    cursor: pointer;
    overflow: hidden;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
    transition: transform 0.3s ease;
}

.gallery-item:hover {
    transform: scale(1.05);
}

.gallery-item img {
    width: 100%;
    height: 200px;
    object-fit: cover;
    display: block;
}

/* Fullscreen image viewer */
.gallery-item:fullscreen {
    width: 100vw;
    height: 100vh;
    margin: 0;
    border-radius: 0;
    display: flex;
    align-items: center;
    justify-content: center;
    background: #000;
    box-shadow: none;
}

.gallery-item:fullscreen img {
    width: 100%;
    height: 100%;
    object-fit: contain;
    max-width: 100vw;
    max-height: 100vh;
}

/* Info overlay in fullscreen */
.image-info {
    position: absolute;
    bottom: 20px;
    left: 20px;
    color: white;
    background: rgba(0,0,0,0.6);
    padding: 15px;
    border-radius: 4px;
```

```
        display: none;
    }

    .gallery-item:fullscreen ~ .image-info {
        display: block;
    }

    /* Backdrop for image viewer */
    .gallery-item::backdrop {
        background-color: rgba(0, 0, 0, 0.95);
    }
```

## 5.3 Example 3: Game Canvas Fullscreen

### 5.3.1 Game Container Styling

```
/* Default game view */
.game-container {
    max-width: 1000px;
    margin: 20px auto;
    background: #222;
    border: 3px solid #444;
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 4px 12px rgba(0,0,0,0.3);
}

canvas {
    display: block;
    width: 100%;
    height: auto;
    background: #000;
}

.game-controls {
    background: #333;
    padding: 15px;
    display: flex;
    gap: 10px;
    justify-content: center;
}

.game-controls button {
    padding: 10px 20px;
    background: #667eea;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
}
```

```
/* Fullscreen game */
.game-container:fullscreen {
  width: 100vw;
  height: 100vh;
  max-width: none;
  margin: 0;
  border: none;
  border-radius: 0;
  position: fixed;
  top: 0;
  left: 0;
}

.game-container:fullscreen canvas {
  width: 100%;
  height: 100%;
}

.game-container:fullscreen .game-controls {
  position: absolute;
  bottom: 0;
  width: 100%;
  margin: 0;
}

.game-container:fullscreen .game-controls button {
  padding: 15px 30px;
  font-size: 16px;
}

/* Hide UI elements in fullscreen */
.game-container:fullscreen .score,
.game-container:fullscreen .level {
  display: none;
}

.game-container::backdrop {
  background-color: #000;
}
```

## 5.4 Example 4: Presentation Mode

### 5.4.1 Fullscreen Presentation Slides

```
/* Default slide view */
.presentation {
  max-width: 900px;
  margin: 20px auto;
  background: white;
  box-shadow: 0 4px 20px rgba(0,0,0,0.2);
  border-radius: 8px;
  overflow: hidden;
}
```



```
}

.slide {
  display: none;
  padding: 40px;
  background: linear-gradient(135deg, #667eea, #764ba2);
  color: white;
  min-height: 600px;
  font-size: 24px;
}

.slide.active {
  display: flex;
  flex-direction: column;
  justify-content: center;
}

.slide h1 {
  font-size: 48px;
  margin-bottom: 20px;
}

.slide-controls {
  background: #f5f5f5;
  padding: 20px;
  display: flex;
  justify-content: space-between;
  gap: 10px;
}

/* Fullscreen presentation */
.presentation:fullscreen {
  width: 100vw;
  height: 100vh;
  max-width: none;
  margin: 0;
  border-radius: 0;
}

.presentation:fullscreen .slide {
  width: 100vw;
  height: 100vh;
  padding: 60px;
  font-size: 32px;
}

.presentation:fullscreen .slide h1 {
  font-size: 64px;
}

.presentation:fullscreen .slide-controls {
  position: absolute;
  bottom: 0;
  width: 100%;
}
```

```
    border-radius: 0;
}

/* Hide progress bar in fullscreen */
.presentation:fullscreen .progress {
    display: none;
}

.presentation::backdrop {
    background-color: rgba(0, 0, 0, 0.95);
}
```

## Chapter 6

# Advanced Techniques

### 6.1 Animations in Fullscreen

#### 6.1.1 Fullscreen Transition Animation

```
.video-player {  
    transition: all 0.3s ease;  
}  
  
.video-player:fullscreen {  
    animation: enterFullscreen 0.5s ease-out;  
}  
  
@keyframes enterFullscreen {  
    from {  
        transform: scale(0.95);  
        opacity: 0;  
    }  
    to {  
        transform: scale(1);  
        opacity: 1;  
    }  
}  
  
@keyframes exitFullscreen {  
    from {  
        transform: scale(1);  
        opacity: 1;  
    }  
    to {  
        transform: scale(0.95);  
        opacity: 0;  
    }  
}
```

## 6.2 Responsive Fullscreen

### 6.2.1 Fullscreen with Media Queries

```
/* Desktop fullscreen */
@media (min-width: 1024px) {
  .video-player:fullscreen {
    aspect-ratio: 16 / 9;
  }
}

/* Mobile fullscreen */
@media (max-width: 768px) {
  .video-player:fullscreen {
    width: 100vw;
    height: 100vh;
  }

  .controls {
    padding: 15px;
  }

  button {
    padding: 8px 16px;
    font-size: 12px;
  }
}

/* Landscape mobile */
@media (max-width: 768px) and (orientation: landscape) {
  .video-player:fullscreen {
    height: 100vh;
  }

  .controls {
    position: fixed;
    bottom: 0;
  }
}
```

## 6.3 Combining with Other Pseudo-classes

### 6.3.1 Interactive Fullscreen Styles

```
/* Fullscreen with hover */
.video-player:fullscreen:hover .controls {
  opacity: 1;
}

/* Fullscreen and focus */
.video-player:fullscreen:focus {
```

```
    outline: 2px solid yellow;
}

/* Fullscreen and active */
button:fullscreen:active {
    transform: scale(0.95);
}

/* Combine multiple states */
video:fullscreen {
    width: 100vw;
    height: 100vh;
}

video:fullscreen::-webkit-media-controls {
    display: flex;
}
```

## 6.4 Keyboard Shortcuts in Fullscreen

### 6.4.1 JavaScript Implementation

```
document.addEventListener('keydown', (e) => {
    if (document.fullscreenElement) {
        switch(e.key) {
            case 'Escape':
                document.exitFullscreen();
                break;
            case ' ':
                // Pause/play video
                e.preventDefault();
                togglePlayPause();
                break;
            case 'ArrowRight':
                // Next slide/video
                e.preventDefault();
                nextItem();
                break;
            case 'ArrowLeft':
                // Previous slide/video
                e.preventDefault();
                previousItem();
                break;
            case 'f':
                // Toggle fullscreen
                e.preventDefault();
                toggleFullscreen();
                break;
        }
    }
});
```

## Chapter 7

# Browser Support

### 7.1 Current Support Status

| Browser           | Version | Status          |
|-------------------|---------|-----------------|
| Chrome/Edge       | v15+    | Full Support    |
| Firefox           | v9+     | Full Support    |
| Safari (macOS)    | 5.1+    | Full Support    |
| Safari (iOS)      | 16.1+   | Full Support    |
| Opera             | v10.6+  | Full Support    |
| Samsung Internet  | v1.0+   | Full Support    |
| Internet Explorer | 11      | Partial Support |

### 7.2 Fallback Strategies

#### 7.2.1 Progressive Enhancement

```
/* Default styles that work everywhere */
.video-player {
  max-width: 100%;
  background: #000;
}

/* Enhanced styles for browsers supporting :fullscreen */
.video-player:fullscreen {
  width: 100vw;
  height: 100vh;
}

/* With vendor prefixes for older browsers */
.video-player:-webkit-full-screen {
  width: 100vw;
  height: 100vh;
}

.video-player:-moz-full-screen {
  width: 100vw;
  height: 100vh;
}
```

```
}
```

# Chapter 8

## Best Practices

### 8.1 Key Recommendations

#### 8.1.1 1. Always Provide Exit Instructions

```
.exit-hint {  
  position: absolute;  
  top: 10px;  
  right: 10px;  
  color: white;  
  background: rgba(0,0,0,0.5);  
  padding: 10px;  
  border-radius: 4px;  
  font-size: 12px;  
  display: none;  
}  
  
.video-player:fullscreen .exit-hint {  
  display: block;  
}
```

**Content:** “Press ESC to exit fullscreen”

#### 8.1.2 2. Optimize Typography for Fullscreen

```
.presentation:fullscreen {  
  font-size: 32px;  
  line-height: 1.6;  
  letter-spacing: 0.5px;  
}  
  
.presentation:fullscreen h1 {  
  font-size: 64px;  
  font-weight: bold;  
}  
  
.presentation:fullscreen h2 {  
  font-size: 48px;  
}
```



```
.presentation:fullscreen p {  
  font-size: 32px;  
  max-width: 90%;  
  margin: 0 auto;  
}
```

### 8.1.3 3. Use Safe Area Insets

```
.video-player:fullscreen {  
  width: 100vw;  
  height: 100vh;  
  padding-top: env(safe-area-inset-top);  
  padding-bottom: env(safe-area-inset-bottom);  
  padding-left: env(safe-area-inset-left);  
  padding-right: env(safe-area-inset-right);  
}
```

### 8.1.4 4. Provide Feedback on Fullscreen State

```
.fullscreen-btn::after {  
  content: '';  
  margin-left: 5px;  
}  
  
.fullscreen-btn.active::after {  
  content: ' (Fullscreen)';  
}  
  
/* JavaScript to toggle active class */  
button.addEventListener('click', function() {  
  this.classList.toggle('active');  
});
```

### 8.1.5 5. Test on Multiple Devices

Test fullscreen on:

- Desktop browsers (Chrome, Firefox, Safari, Edge)
- Tablets with various screen sizes
- Mobile phones (with and without notches)
- Different aspect ratios (4:3, 16:9, 21:9, etc.)
- Portrait and landscape orientations

## Chapter 9

# Common Issues & Solutions

### 9.1 Issue 1: Fullscreen Request Denied

**Problem:** User clicks fullscreen but nothing happens.

**Causes:**

- Not triggered by user gesture
- Not on HTTPS connection
- Browser permissions denied
- Element not eligible (e.g., cross-origin iframe)

**Solution:**

```
button.addEventListener('click', async () => {
  try {
    await element.requestFullscreen();
  } catch (err) {
    console.error(`Error: ${err.name} - ${err.message}`);

    // Provide user feedback
    if (err.name === 'NotSupportedError') {
      alert('Fullscreen not supported');
    } else if (err.name === 'NotAllowedError') {
      alert('Fullscreen permission denied');
    }
  }
});
```

### 9.2 Issue 2: Styling Not Applied

**Problem:** CSS :fullscreen styles not working.

**Common Causes:**

- Missing vendor prefixes
- Selector specificity too low

- Using wrong pseudo-class name
- Element not actually in fullscreen

**Solution:**

```
/* Include all vendor prefixes */
.element:fullscreen,
.element:-webkit-full-screen,
.element:-moz-full-screen,
.element:-ms-fullscreen {
    width: 100vw;
    height: 100vh;
    background: #000;
}

/* Use higher specificity if needed */
.container .element:fullscreen {
    /* More specific selector */
}
```

### 9.3 Issue 3: Controls Disappear in Fullscreen

**Problem:** Important UI elements hidden when fullscreen.

**Solution:**

```
/* Ensure important elements stay visible */
.video-player:fullscreen .essential-controls {
    display: flex !important;
    position: fixed;
    bottom: 20px;
    z-index: 1000;
}

/* Use overlay pattern for optional controls */
.video-player:fullscreen .optional-controls {
    opacity: 0;
    transition: opacity 0.3s;
}

.video-player:fullscreen:hover .optional-controls {
    opacity: 1;
}
```

### 9.4 Issue 4: Mobile Fullscreen Problems

**Problem:** Different behavior on iOS vs Android.

**Solution:**

```
@media (max-width: 768px) {
    .video-player:fullscreen {
        width: 100vw;
    }
}
```

```
    height: 100vh;
    position: fixed;
    top: 0;
    left: 0;
  }

  /* Account for status bar */
  @supports (padding: max(0px)) {
    .video-player:fullscreen {
      padding: env(safe-area-inset-top)
        env(safe-area-inset-right)
        env(safe-area-inset-bottom)
        env(safe-area-inset-left);
    }
  }
}
```

## Chapter 10

# Advanced Use Cases

### 10.1 Multi-Element Fullscreen

#### 10.1.1 Managing Multiple Fullscreen Elements

```
/* Only one element can be fullscreen at a time */
.element1:fullscreen {
  z-index: 1001;
}

.element2:fullscreen {
  z-index: 1001;
}

/* Backdrop styling for all fullscreen elements */
:fullscreen::backdrop {
  background-color: rgba(0, 0, 0, 0.95);
}
```

#### 10.1.2 JavaScript Coordination

```
async function enterFullscreen(element) {
  // Exit current fullscreen if any
  if (document.fullscreenElement) {
    await document.exitFullscreen();
  }

  // Enter new fullscreen
  await element.requestFullscreen();
}

// Usage
button1.addEventListener('click', () => {
  enterFullscreen(video1);
});

button2.addEventListener('click', () => {
  enterFullscreen(video2);
});
```

```
});
```

## 10.2 Nested Fullscreen Elements

**Important:** Only the top-level fullscreen element can be styled with `:fullscreen`. Nested elements cannot be fullscreen.

```
/* Parent in fullscreen */
.parent:fullscreen {
  width: 100vw;
  height: 100vh;
}

/* Children of fullscreen parent */
.parent:fullscreen .child {
  /* Styles for children of fullscreen element */
  width: 100%;
  height: 100%;
}

/* Cannot do: child:fullscreen (not valid) */
```

# Chapter 11

## Summary & Quick Reference

### 11.1 Key Takeaways

#### 11.1.1 When to Use :fullscreen

Use the :fullscreen pseudo-class when:

- Building video players
- Creating gaming experiences
- Building presentation software
- Designing immersive web apps
- Need to adapt UI for full viewport

#### 11.1.2 When NOT to Use

Don't use :fullscreen when:

- Only supporting very old browsers
- The styling difference is minimal
- Not actually using the Fullscreen API
- Trying to force fullscreen without user interaction

### 11.2 Pseudo-class Reference

| Pseudo-class         | Purpose               | Browser Support |
|----------------------|-----------------------|-----------------|
| :fullscreen          | Element in fullscreen | All modern      |
| :-webkit-full-screen | Webkit version        | Chrome, Safari  |
| :-moz-full-screen    | Firefox version       | Firefox         |
| ::backdrop           | Fullscreen backdrop   | All modern      |

## 11.3 Common Properties in Fullscreen

| Property   | Typical Fullscreen Value      |
|------------|-------------------------------|
| width      | 100vw                         |
| height     | 100vh                         |
| margin     | 0                             |
| padding    | 0 (or env(safe-area-inset-*)) |
| border     | none                          |
| background | #000 or black                 |
| position   | fixed (if needed)             |
| z-index    | High value                    |

## 11.4 Complete CSS Template

```
/* Default state */
.fullscreen-element {
  max-width: 800px;
  background: white;
  margin: 20px auto;
  border-radius: 8px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

/* Modern browsers */
.fullscreen-element:fullscreen {
  width: 100vw;
  height: 100vh;
  max-width: none;
  margin: 0;
  border-radius: 0;
  background: #000;
  display: flex;
  align-items: center;
  justify-content: center;
}

/* Webkit browsers */
.fullscreen-element:-webkit-full-screen {
  width: 100vw !important;
  height: 100vh !important;
}

/* Firefox */
.fullscreen-element:-moz-full-screen {
  width: 100vw !important;
  height: 100vh !important;
}

/* Backdrop styling */
.fullscreen-element::backdrop {
  background-color: rgba(0, 0, 0, 0.95);
}
```



```
}
```

**Thank you for reading!**

For more information, visit [MDN Web Docs](#) and [web.dev](#)